

~/portfolio \$ whoami

Hamed Abdullah

< MERN Stack Developer />

• hamedabdullahmachrur@gmail.com

• +8801765-868671

• Banskhali, Chattogram, Bangladesh

// CAREER OBJECTIVE

Passionate MERN Stack Developer with hands-on experience building responsive web applications. Skilled in React, Node.js, and MongoDB. Seeking an opportunity to contribute to a dynamic team and grow as a full-stack engineer.

// WORK EXPERIENCE

Tech Doctor / Web Developer

Part-time

Eco Mart • Banskhali, Chattogram, BD • Nov 2025 – Present

- ▶ Developed and launched a responsive, modern e-commerce website with a focus on clean UI and smooth user experience.
- ▶ Designed and implemented key frontend features to improve navigation, usability, and overall performance.
- ▶ Managed and maintained the live platform, ensuring stability, reliability, and consistent updates.
- ▶ Identified and resolved technical issues, optimizing performance and reducing bugs.
- ▶ Collaborated with business stakeholders to implement new features and enhance customer experience.

// EDUCATION DETAILS

LINKS



GitHub
hamedabdullahmachrur



Portfolio
netlify.app



Facebook
hamedabdullahmachrur

SKILLS

→ Proficient

- HTML
- CSS
- Tailwind CSS
- JavaScript
- React
- REST API
- Firebase
- Node.js
- Express.js
- MongoDB

→ Currently Learning

- TypeScript
- Next.js
- PostgreSQL & Prisma

Next Level Web Development

Programming Hero · Apr 2026 – Present

Advanced full-stack program covering modern technologies and industry practices.

- TypeScript
- Next.js
- Node.js
- PostgreSQL
- Prisma
- Docker
- Nginx
- AWS
- AI Integration
- SDLC & Agile
- TDD
- CI/CD

Web Development

Programming Hero · Jun 2024 – Dec 2024

Comprehensive full-stack course with real-world projects and strong development foundations.

- HTML
- CSS
- JavaScript
- React
- Node.js
- Express.js
- MongoDB
- Firebase

- Artificial Intelligence
- Docker & Nginx
- AWS
- Software Eng. & Testing

LOCATION

 Banskhali, Chattogram
Bangladesh

```
const User = mongoose.model('User', userSchema);

// Custom Hook
const useProducts = (category, page = 1, limit = 10) => {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch(`/api/products/cat-${category}`)
      .then(r => r.json())
      .then(setData)
      .catch(setError);
  }, [category]);

  return { data, error };
};

// Auth Middleware
const verifyToken = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Unauthorized' });
  jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
    if (err) return res.status(403).json({ error: 'Invalid token' });
    req.user = decoded;
    next();
  });
};

async function getUserById(id) {
  try {
    const user = await User.findById(id);
    return user;
  } catch (error) {
    throw new Error('User not found');
  }
}

// React Component
import { useState, useEffect } from 'react';
import axios from 'axios';

const ProductList = () => {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    axios.get('/api/products')
      .then(res => setProducts(res.data))
      .catch(err => console.error(err));
  }, []);

  return (
    <div>
      {loading ? <div>Loading...</div> : (
        <div>
          {products.map(product => (
            <div>
              {product.name}
              {product.description}
            </div>
          ))}
        </div>
      )}
    </div>
  );
};

// Register Route
router.post('/register', async (req, res) => {
  const { name, email, password } = req.body;
  const hash = await bcrypt.hash(password, 12);
  const user = await User.create({ name, email, password: hash });
  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET);
  res.status(201).json({ user, token });
});

// Login Route
router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user) return res.status(404).json({ error: 'Not found' });
  const match = await bcrypt.compare(password, user.password);
  if (!match) return res.status(401).json({ error: 'Invalid' });
  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET);
  res.json({ user, token });
});

// Dockerfile
FROM node:18-alpine
WORKDIR /app
COPY package.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]
```